

openTSDB安装记录

openTSDB安装记录

- 前期准备
- JDK安装
- zookeeper安装
- hadoop安装
 - 1. 安装过程
 - 2. hadoop HA高可用配置
- hadoop集群初始化
- 设置启动脚本
- 运行过程
- Hbase安装

前期准备

设置apt的国内源

```
sudo vim /etc/apt/sources.list
# 将其修改为如下内容
deb https://mirrors.ustc.edu.cn/ubuntu/ focal main restricted universe
multiverse
deb https://mirrors.ustc.edu.cn/ubuntu/ focal-updates main restricted universe
multiverse
deb https://mirrors.ustc.edu.cn/ubuntu/ focal-backports main restricted universe
multiverse
deb https://mirrors.ustc.edu.cn/ubuntu/ focal-security main restricted universe
multiverse
```

设置root密码

```
sudo passwd root
```

设置用户sudo免密码

```
sudo visudo
# 在末尾添加
sd1m    ALL=(ALL:ALL) NOPASSWD:ALL
```

设置host名

```
# 分别在3台机器设置主机名（hadoop1/hadoop2/hadoop3）
sudo hostnamectl set-hostname hadoop1 # hadoop2/hadoop3同理

# 编辑hosts文件（3台机器均添加以下内容）
sudo vim /etc/hosts
# 添加：
192.168.120.131 hadoop1
192.168.120.132 hadoop2
192.168.120.133 hadoop3
```

设置ssh免密登录

```
# 所有主机都安装ssh服务器
sudo apt install openssh-server -y

# 在hadoop1生成密钥（一路回车）
ssh-keygen -t rsa

# 复制公钥到3台机器（包括自身）
ssh-copy-id hadoop1
ssh-copy-id hadoop2
ssh-copy-id hadoop3
```

在hadoop1中设置文件分发脚本

```
## xsync 脚本代码
## 使用方法：xsync [分发文件或者文件夹]
#!/bin/bash
#1. 判断参数个数
if [ $# -lt 1 ]
then
    echo Not Enough Argument!
    exit;
fi
#2. 遍历集群所有机器
for host in hadoop1 hadoop2 hadoop3
do
    echo ===== $host =====
    #3. 遍历所有目录，挨个发送
    for file in $@
    do
        #4. 判断文件是否存在
        if [ -e $file ]
        then
            #5. 获取父目录
            pdir=$(cd -P $(dirname $file); pwd)
            #6. 获取当前文件的名称
            fname=$(basename $file)
            ssh $host "mkdir -p $pdir"
            rsync -av $pdir/$fname $host:$pdir
        else
            echo $file does not exists!
        fi
    done
done
```

```
## 设置为可执行权限
chmod +x xsync

## 复制到/bin下，可以全局使用
sudo cp xsyn /bin
```

下载安装包

```
# jdk版本
jdk-8u471-linux-x64.tar.gz

# 清华源镜像网址
https://mirrors.ustc.edu.cn/apache/

# 下载如下版本
apache-zookeeper-3.7.2-bin.tar.gz
hadoop-3.3.5.tar.gz
hbase-2.4.18-bin.tar.gz
```

JDK安装

解压JDK，并配置环境变量

```
tar -zxvf jdk-8u471-linux-x64.tar.gz -C /opt/module/

# 添加环境变量
vim /etc/profile.d/opentsdb_env.sh
# 添加如下内容
# JAVA_HOME
export JAVA_HOME=/opt/module/jdk1.8.0_471
export PATH=$PATH:$JAVA_HOME/bin

source /etc/profile

# 验证
java -version
```

zookeeper安装

以hadoop1主机为例，新建/opt/module目录

```
cd /opt
mkdir module
# 修改module目录的所在组
sudo chown sdlm:sdlm /opt/module
```

将zookeeper解压到/opt/module下

```
tar -zxvf apache-zookeeper-3.7.2-bin.tar.gz -C /opt/module
cd /opt/module
mv apache-zookeeper-3.7.2-bin zookeeper-3.7.2
```

对zookeeper进行配置

```
cd /opt/module/zookeeper-3.7.2
# 新建存储文件夹
mkdir zkData
# 修改zoo.conf
cd ./conf
cp zoo_sample.cfg zoo.cfg
vim zoo.cfg
# 主要修改如下内容
dataDir=/opt/module/zookeeper-3.7.2/zkData # 存储目录
server.1=hadoop1:2888:3888 # zookeeper主机1, 其中hadoop1为主机名称
server.2=hadoop2:2888:3888
server.3=hadoop3:2888:3888
```

将上述zookeeper文件夹复制到hadoop2和hadoop3中

配置节点ID

```
# 设置id, 每个主机设置不同的id号
# hadoop1为1 hadoop2为2 hadoop3为3
# hadoop1
echo "1" > /opt/module/zookeeper-3.7.2/zkData/myid
# hadoop2
echo "2" > /opt/module/zookeeper-3.7.2/zkData/myid
# hadoop3
echo "3" > /opt/module/zookeeper-3.7.2/zkData/myid
```

启动zookeeper

```
# 在hadoop1、hadoop2、hadoop3中分别启动即可
/opt/module/zookeeper-3.7.2/bin/zkServer.sh start

# 查看状态, 它们会根据启动的顺序为 leader 或者 follower
/opt/module/zookeeper-3.7.2/bin/zkServer.sh status
```

hadoop安装

1. 安装过程

在hadoop1中, 解压安装包

```
tar -zxvf hadoop-3.3.5.tar.gz -C /opt/module/
```

配置环境变量

```
# 在opentsdb_env.sh中增加hadoop路径
# 添加环境变量
vim /etc/profile.d/opentsdb_env.sh
# 添加如下内容
# HADOOP_HOME
export HADOOP_HOME=/opt/module/hadoop-3.3.5
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

配置 Hadoop 核心文件

```
vim /opt/module/hadoop-3.3.5/etc/hadoop/hadoop-env.sh
# 添加如下内容
export JAVA_HOME=/opt/module/jdk1.8.0_471/
export HADOOP_HOME=/opt/module/hadoop-3.3.5/
```

hadoop四个配置文件core-site.xml、hdfs-site.xml、yarn-site.xml、mapred-site.xml的作用：

- core-site.xml：核心全局配置。配置 Hadoop 公共参数，指定 **HDFS 名称节点 (NameNode) 地址**，设置临时目录、权限、I/O 缓冲等通用配置；
- hdfs-site.xml：HDFS 专用配置。配置 HDFS 自身参数，设置 **文件块副本数**，配置 NameNode、DataNode 数据存放目录，权限检查、访问控制、HDFS 缓存等；
- yarn-site.xml：YARN 资源调度配置。配置 **ResourceManager、NodeManager**，设置容器内存、CPU 限制，配置日志聚集、节点心跳、调度器类型等；
- mapred-site.xml：MapReduce 计算框架配置。配置 MapReduce 运行环境，指定 MR 运行在 **YARN 上**，设置 Map/Reduce 任务内存、CPU、日志等。

2. hadoop HA高可用配置

按照如下的方式进行部署配置

	hadoop1	hadoop2	hadoop3
	zookeeper	zookeeper	zookeeper
HDFS	NameNode	NameNode	
HDFS	DataNode	DataNode	DataNode
YARN		ResourceManager	ResourceManager
YARN	NodeManager	NodeManager	NodeManager

配置core-site.xml，在原来的core-site.xml中的内推荐property的配置

```
<configuration>
  <!-- 整个集群的名字 -->
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://mycluster</value>
  </property>
  <!-- 指定 hadoop 数据的存储目录 -->
```

```

<property>
  <name>hadoop.tmp.dir</name>
  <value>/opt/module/hadoop-3.3.5/data</value>
</property>
<!-- 配置 HDFS 网页登录使用的静态用户，不设置会导致web UI操作无权限 -->
<property>
  <name>hadoop.http.staticuser.user</name>
  <value>sd1m</value>
</property>
<!-- ZooKeeper地址 -->
<property>
  <name>ha.zookeeper.quorum</name>
  <value>hadoop1:2181,hadoop2:2181,hadoop3:2181</value>
</property>
</configuration>

```

配置hdfs-site.xml

```

<configuration>
  <!-- 副本数 -->
  <property>
    <name>dfs.replication</name>
    <value>3</value>
  </property>
  <!-- 集群名称，和 core-site 保持一致 -->
  <property>
    <name>dfs.nameservices</name>
    <value>mycluster</value>
  </property>

  <!-- 两个 NameNode 节点名称 -->
  <property>
    <name>dfs.ha.namenodes.mycluster</name>
    <value>nn1,nn2</value>
  </property>
  <!-- nn1 = hadoop1 -->
  <property>
    <name>dfs.namenode.rpc-address.mycluster.nn1</name>
    <value>hadoop1:8020</value>
  </property>
  <property>
    <name>dfs.namenode.http-address.mycluster.nn1</name>
    <value>hadoop1:9870</value>
  </property>
  <!-- nn2 = hadoop2 -->
  <property>
    <name>dfs.namenode.rpc-address.mycluster.nn2</name>
    <value>hadoop2:8020</value>
  </property>
  <property>
    <name>dfs.namenode.http-address.mycluster.nn2</name>
    <value>hadoop2:9870</value>
  </property>

  <!-- JournalNode 共享日志（自动同步元数据） -->
  <property>
    <name>dfs.namenode.shared.edits.dir</name>

```

```

<value>qjournal://hadoop1:8485;hadoop2:8485;hadoop3:8485/mycluster</value>
</property>
<!-- 设置JournalNode 日志文件路径 -->
<property>
  <name>dfs.journalnode.edits.dir</name>
  <value>/opt/module/hadoop-3.3.5/tmp/journal</value>
</property>
<!-- 故障自动切换代理 -->
<property>
  <name>dfs.client.failover.proxy.provider.mycluster</name>

  <value>org.apache.hadoop.hdfs.server.namenode.ha.ConfiguredFailoverProxyProvide
r</value>
</property>

<!-- 生产环境HDFS HA隔离配置（推荐） -->
<property>
  <name>dfs.ha.fencing.methods</name>
  <!-- 优先电源隔离，失败则兜底切换 -->
  <value>shell(/bin/true)</value>
</property>
<!-- 隔离机制（防止双主），使用ssh隔离有点问题，当主设备直接关机了，就会导致ssh失败，导致
备用设备启动失败-->
<!-- 生产环境HDFS HA隔离配置（推荐），默认返回true。一般是远程给主节点远程关机 -->
<property>
  <name>dfs.ha.fencing.methods</name>
  <value>shell(/bin/true)</value>
</property>

<!-- 开启自动故障恢复 -->
<property>
  <name>dfs.ha.automatic-failover.enabled</name>
  <value>true</value>
</property>
</configuration>

```

配置yarn-site.xml

```

<configuration>
<!-- Site specific YARN configuration properties -->
<!-- 指定 MR 走 shuffle -->
<property>
  <name>yarn.nodemanager.aux-services</name>
  <value>mapreduce_shuffle</value>
</property>
<!-- 启用 ResourceManager HA 功能 -->
<property>
  <name>yarn.resourcemanager.ha.enabled</name>
  <value>true</value>
</property>
<property>
  <name>yarn.resourcemanager.cluster-id</name>
  <value>yarncluster</value>
</property>
<property>
  <name>yarn.resourcemanager.ha.rm-ids</name>

```

```

        <value>rm1,rm2</value>
    </property>
</property>
    <name>yarn.resourcemanager.hostname.rm1</name>
    <value>hadoop3</value>
</property>
<property>
    <name>yarn.resourcemanager.hostname.rm2</name>
    <value>hadoop2</value>
</property>
<!-- 设置访问网址和端口 -->
<property>
    <name>yarn.resourcemanager.webapp.address.rm1</name>
    <value>hadoop3:8088</value>
</property>
<property>
    <name>yarn.resourcemanager.webapp.address.rm2</name>
    <value>hadoop2:8088</value>
</property>
<property>
    <name>yarn.resourcemanager.zk-address</name>
    <value>hadoop1:2181,hadoop2:2181,hadoop3:2181</value>
</property>
<!-- 启用重启 ResourceManager 后保留其恢复状态的功能 -->
<property>
    <name>yarn.resourcemanager.recovery.enabled</name>
    <value>true</value>
</property>
</configuration>

```

配置mapred-site.xml

```

<configuration>
    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>
</configuration>

```

配置workers，workers里面负责指定谁为dataNode。假设我只需要指定hadoop1和hadoop3为DataNode，则下述文件中删除hadoop2，然后修改hdfs-site.xml中的副本数为2即可。

```

vim workers
# 添加如下内容
hadoop1
hadoop2
hadoop3

```

将 `/etc/profile.d/opentsdb_env.sh` 和 `/opt/module/hadoop-3.3.5/` 分发给hadoop2和hadoop3。

hadoop集群初始化

```
# 启动zookeeper，每台机器都执行
/opt/module/zookeeper-3.7.2/bin/zkServer.sh start
# 可以查看一下状态，正常两个follower，一个leader
/opt/module/zookeeper-3.7.2/bin/zkServer.sh status

# 启动JournalNode，每台都执行
hdfs --daemon start journalnode

# 只对 hadoop1 执行格式化ZK
hdfs zkfc -formatZK

# 只对 hadoop1 执行格式化Namenode
hdfs namenode -format

## 以下操作应该不是必须的
# 启动 hadoop1 的NameNode
hdfs --daemon start namenode

# hadoop2 同步元数据
hdfs namenode -bootstrapStandby

# 启动hadoop1和hadoop2中的 ZKFC
hdfs --daemon start zkfc

# 在hadoop中启动HDFS和YARN
start-dfs.sh
start-yarn.sh
```

设置启动脚本

在主设备中设置一键启动和停止脚本，以下的 `/opt/module/zookeeper-3.7.2` 和 `/opt/module/hadoop-3.3.5` 以具体的路径为准：

```
## start-hadoop-ha.sh
#!/bin/bash
echo "=====
echo "      Hadoop HA 集群 一键启动脚本      "
echo "      主节点自动控制所有节点      "
echo "=====

# ===== 【必须修改：你的3台机器名】 =====
nodes="hadoop1 hadoop2 hadoop3"
namenode_nodes="hadoop1 hadoop2"
# =====

echo -e "\n☑ 1. 启动所有节点 zookeeper..."
for node in $nodes; do
    ssh $node "/opt/module/zookeeper-3.7.2/bin/zkServer.sh start"
done

echo -e "\n☑ 2. 一键启动 HDFS (NameNode、DataNode、JournalNode、
DFSZKFailoverController) ..."
```

```

/opt/module/hadoop-3.3.5/sbin/start-dfs.sh

echo -e "\n☑ 3. 一键启动 YARN (ResourceManager、NodeManager) ..."
/opt/module/hadoop-3.3.5/sbin/start-yarn.sh

echo -e "\n=====
echo "                启动完成!                "
echo "=====
jps

## stop-hadoop-ha.sh
#!/bin/bash
echo "停止 Hadoop HA 集群..."

nodes="hadoop1 hadoop2 hadoop3"

/opt/module/hadoop-3.3.5/sbin/stop-yarn.sh
/opt/module/hadoop-3.3.5/sbin/stop-dfs.sh

for node in $nodes; do
    ssh $node "/opt/module/zookeeper-3.7.2/bin/zkServer.sh stop"
done

echo "集群已全部停止! "

```

在启动脚本中远程启动zkServer.sh如果报错找不到JAVA_HOME，则需要
在 /opt/module/zookeeper-3.7.2/bin/zkEnv.sh 中的文件首部加入 export
JAVA_HOME=/opt/module/jdk1.8.0_471，每台设备都需要设置。

运行成功后查看NameNode和ResourceManager的主备状态。

```

# 查看所有 NameNode 状态
hdfs haadmin -getAllServiceState

# 查看所有 RM 状态
# 实际可能会出现yarn与系统中的npm中的yarn指令冲突，需要加上yarn的路径，如
# /opt/module/hadoop-3.3.5/bin/yarn radmin -getAllServiceState
yarn radmin -getAllServiceState

```

运行过程

在hadoop1中运行

```

# 初始化NameNode（仅首次执行）
hdfs namenode -format

# 启动HDFS
start-dfs.sh

```

在hadoop2中运行

```
# 因为在yarn-site.xml中配置了ResourceManager为hadoop2
# 启动YARN
start-yarn.sh
```

访问 Web 界面：`http://hadoop1:9870` (HDFS) 、 `http://hadoop2:8088` (YARN) ， 能够正常访问代表hadoop部署成功

Hbase安装

在hadoop1中，解压安装包

```
tar -zxvf hbase-2.4.18-bin.tar.gz -C /opt/module/
cd /opt/module
mv hbase-2.4.18 hbase
```

配置环境变量

```
sudo vim /etc/profile.d/opentsdb_env.sh
# 添加如下内容
# HBASE_HOME
export HBASE_HOME=/opt/module/hbase
export PATH=$PATH:$HBASE_HOME/bin
```